Université Grenoble Alpes
Licence d'Informatique L3

Introduction aux Systèmes et Réseaux

Séance 1 : Découverte des processus Unix

Objectifs de la séance

Pendant cette séance vous allez travailler avec les *processus*. L'objectif est de comprendre la notion de processus et de découvrir différentes manières d'observer, créer et contrôler les processus.

Dans la suite, lisez, apprenez et exécutez séquentiellement les différentes parties. **Notez les réponses aux différentes questions.**

1 Définition des processus

Le système d'exploitation est le logiciel qui permet d'utiliser la machine (le matériel). Tout ce que le système fait est fait par des *processus*.

> **À SAVOIR : Processus**
>
> Un processus est un programme en exécution.

Quand vous démarrez un ordinateur, la première chose qu'il fait et de *boot*-er : cela consiste à lancer le système d'exploitation. Le lancement du système d'exploitation consiste en le lancement des *processus système* i.e. les processus qui se chargent d'exécuter les traitements correspondant au fonctionnement du système d'exploitation.

Pour se distinguer, les processus ont des numéros *uniques*.

> **À SAVOIR : Numéro de processus (`PID`)**
>
> Chaque processus a un numéro (entier positif) qui est un identifiant unique. On parle de PID d'un processus ce qui vient du nom anglais **`Process IDentifier`**.

Le premier processus du système a le `PID 1`.

> **Voir le premier processus du système**
>
> Pour voir l'existence du processus avec **`PID 1`**, exécuter la commande suivante
> **`ps -ax | head -n 2`**

Explication

Cette commande a deux parties :

1)) Une première partie qui utilise la commande **ps**.
- **ps** montre la liste des tous les processus qui tournent.
- les différentes options de cette commande permettent de filtrer et de montrer différents détails sur les processus.
- la documentation complète de la commande est données par **man ps**

2) La deuxième partie de la commande permet de ne prendre que les 2 premières lignes de la sortie de **ps -ax**.

Vous deviez voir une sortie qui ressemble à :

```
PID TTY           TIME CMD
    1 ??       316:51.05 /sbin/launchd
```

2 Observer les processus et leurs relations

Exécuter et observer

Vous travaillez dans une fenêtre, un terminal.
Essayez dans ce terminal simplement la commande
ps

Quels processus voyez-vous ?

Savez-vous ce qu'est le processus **bash** ?

Quel est le PID du **bash** ?

Lancez maintenant
gedit &

Refaites la commande **ps**. Que voyez-vous ?

Quel est le PID du **gedit** ?

Terminez le processus **gedit** avec la commande
kill -9 mettre_ici_PID_gedit

Relancez **gedit** avec
gedit &

Quel est le PID du **gedit** maintenant ? Que pouvez-vous conclure ?

Explication

Quand vous ouvrez un terminal, il lance en premier un processus **`shell`** qui est l'interpréteur de commandes. C'est le processus qui récupère ce que vous tapez et qui se charge de l'exécution des commandes correspondantes.

Le processus **`bash`** (ou possiblement **`tcsh`** ou **`zsh`**...) est donc l'interpréteur de commandes.

Après le lancement de **`gedit`**, vous devriez observer l'apparition d'un nouveau processus qui correspond à l'exécution de l'éditeur.
Chaque lancement de l'éditeur correspond à la création d'un nouveau processus qui a donc un numéro différent.

À SAVOIR : Processus père

A part le premier processus système, tout processus est créé par un autre processus qui est son processus *père*.

Exécuter et observer

Pour avoir plus d'informations sur les processus, essayez la commande
`ps -f`

Dans la sortie vous devez observer deux champs : **`PID`** et **`PPID`**. Le **`PID`** est, comme on l'a vu, le numéro du processus. Le **`PPID`** est le numéro du processus qui a créé ce processus : c'est le processus père.

Exécuter et observer

Sur la sortie de la commande

Quel est le **`PPID`** de **`gedit`** ?
A quel processus correspond ce numéro ?
Que pouvez-vous conclure ?

COMPLETER cette définition – À SAVOIR : Shell

... à vous ...

Exécuter et observer une autre visualisation des processus

Tester la commande
`pstree -u votre_nom_de_login`

À SAVOIR : Interface en ligne de commande du système d'exploitation

Quand vous travaillez dans un terminal et tapez des commandes, vous demandez des choses au système d'exploitation. Vous travaillez en *ligne de commande*. Les fonctions qui vous sont accessibles composent l'*interface en ligne de commande* du système d'exploitation.

À SAVOIR : Interface graphique du système d'exploitation

Quand vous cliquez pour lancer une application sur votre machine, visualiser des fichiers, etc., vous demandez des choses au système d'exploitation mais via son *interface graphique*.

Exécuter et observer

Tester la commande **`ps -a`**

Tester la commande **`top`**

Qu'observez-vous ?

3 Interface programmatique du système d'exploitation

Dans les sections précédentes vous avez créé des processus en cliquant ou en tapant des commandes en ligne de commande.

Dans cette section, nous allons voir que nous pouvons *programmer* la création et le contrôle de processus. Nous allons donc utiliser l'*interface programmatique du système d'exploitation*.

3.1 Création de processus : : la fonction `fork`

Nous allons commencer avec le petit programme suivant (`fork1.c`, fourni) :

```
int main() {                                                    // début du processus "main"
  pid_t pid;                                                    // |
                                                                // |
  pid = fork();                                                 // |___ création d'un fils
                                                                // |
  if (pid == -1) { exit(-3);}                                   // |     création échouée
                                                                // |   | deux processus en exécution
  else if (pid > 0) {                                           // |     test OK pour main = père
    printf("je suis le père, mon PID est %d\n", getpid());// |     exécuté par le père
  } else {                                                      //     | test OK pour le fils
    printf("je suis le fils, mon PID est %d\n", getpid());//     | exécuté par le fils
  }                                                             // |   |
}                                                               // |   | terminaison de processus
```

À SAVOIR : La fonction `getpid`

La fonction `gepid()` retourne le PID du processus courant.

À SAVOIR : La fonction `fork`

La fonction `fork()` crée un processus "fils" du processus appelant (le "père"), avec le même programme que ce dernier. La valeur renvoyée par `fork()` est :

- au père : le numéro (PID) du processus fils.
- au fils : 0.

En cas d'échec (limite sur le nombre de processus, manque de mémoire, ...), aucun processus n'est créé, et la valeur -1 est renvoyée au processus appelant.

À SAVOIR : Compilation de base

La commande

`gcc -o prog prog1.c ...progn.c`,

où `prog1, ...progn` sont des programmes en C, compile et lie ces programmes pour produire un exécutable `prog`, qu'on peut lancer par la commande `./prog`.

Exécuter et observer

Compiler et exécuter le programme `fork1.c`.

Observer et comprendre ce qui se passe.

Exécuter plusieurs fois : que constatez-vous au niveau des PID ?

Vous avez observé un programme qui crée deux processus qui s'exécutent en parallèle et de manière indépendente. Ils exécutent **le même code** mais ont chacun une copie propre de ce code. C'est ce que nous allons observer avec le programme `fork_variable.c`, qui vous est fourni.

Exécuter et observer

Compiler et exécuter le programme `fork_variable.c`. Ici la différence est l'existence d'une variable locale `x`.

Observer et comprendre ce qui se passe.
Pourquoi les deux valeurs de `x` sont-elles différentes ?

Dans la suite nous allons travailler avec la bibiothèque CSAPP qui facilite énormément l'utilisation de l'interface programmatique du système.

Bibliothèque CSAPP

Pendant tout le travail en SR, nous allons utiliser la bibliothèque CSAPP. Elle vous est fournie via les deux fichiers **csapp.h** (déclarations de fonctions) et **csapp.c** (implantations).

La bibliothèque fournit des *wrappers* des fonctions de base pour simplifier leur utilisation. Ainsi, à la place de **fork**, nous allons utiliser **Fork** dont le code est le suivant :

```
pid_t Fork(void)
{
    pid_t pid;

    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}
```

La fonction `unix_error` affiche un message d'erreur et termine l'exécution du programme.

À SAVOIR : Compilation avec CSAPP

Pour utiliser la bibliothèque CSAPP vous devez compiler votre programme avec `csapp.c` et inclure (`#include`) `csapp.h`. Nous vous fournissons un **Makefile** en guise d'exemple. Dans ce Makefile, le règles de compilation par défaut sont désactivés [a].

Pour compiler un programme `progMoi.c` et obtenir un exécutable `progMoi` (en utilisant la bibliothèque csapp), il vous suffit de faire

`make progMoi`.

a. Vous pouvez les réactiver et tester... De manière générale nous vous conseillons fortement d'adopter les Makefiles qui vous faciliteront la vie dans le futur :) N'hésitez pas à poser des questions sur leur utilisation, utilité et syntaxe !

Exécuter et observer

Regardez le programme **fork_variable_csapp.c** qui est la version modifiée de `fork_variable.c` avec l'utilisation de CSAPP.

Comprendre le code, compiler et exécuter.

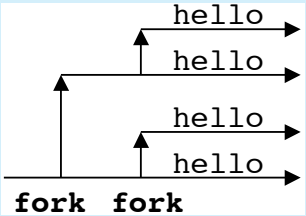
Exécuter et observer

Compiler et exécuter le programme **fork_boucle.c**.

Combien de hello sont affichés ? Expliquer ce qui se passe.

Explication

Le schéma qui représente la création des différents processus est le suivant :



1. Le premier processus, celui qui exécute la fonction `main` rentre dans la boucle avec `i=0`.
2. Il exécute la fonction **`Fork`** et crée un processus fils. A partir d'ici, deux processus existent et s'exécutent en parallèle. Ils exécutent tous les deux la suite du code.
3. les deux processus vont donc continuer, chacun de son coté, à exécuter la boucle. Pour chacun sa variable `i` va être égale à 1.
4. Les deux processus vont donc exécuter chacun la fonction **`Fork`** et créer chacun un processus fils.
5. a partir d'ici quatre processus existent et continue avec l'exécution du reste des instructions.
6. ils font rerentrer dans la boucle mais leurs `i` respectifs vont être égaux à 2 et donc ils vont sortir de la boucle.
7. en sortant de la boucle ils vont écrire `hello`

Attention Dans le schéma, le deuxième appel à **`Fork`** est représenté de manière aligné i.e. comme si les deux processus appelaient la fonction en même temps. Ceci n'est absolument pas vrai dans le cas général et ici est fait par souci de simplicité.

3.2 La fonction `exit`

Vous avez vu dans plusieurs des exemples précédents, l'utilisation de la fonction **`exit`** que vous connaissez certainement déjà.

À SAVOIR : La fonction **`exit`**

La fonction **`exit`** termine le processus appelant.

Si le processus qui exécute la fonction `main` n'appelle pas **`exit`** explicitement, la fonction est appelée automatiquement à la fin de la fonction.

Le paramètre de la fonction définit le statut de terminaison du processus. Par convention, en C, un statut $= 0$ indique que l'exécution s'est déroulée sans erreurs, alors qu'un statut < 0 indique un problème.

Exécuter et observer

Compiler le programme **`testfork.c`**.

Exécuter la commande
`./testfork & ps`
Qu'observez-vous ?

4 Attente entre processus père et processus fils

À SAVOIR : Attente de fils et zombis

Un processus père peut attendre la fin d'un ou plusieurs fils en utilisant **`wait()`** ou **`waitpid()`**.

Tant que son père n'a pas pris connaissance de sa terminaison par l'une de ces primitives, un processus terminé reste dans un état dit *zombi*.

Un processus zombi ne peut plus s'exécuter, mais consomme encore des ressources. *Il faut éviter de conserver des processus dans cet état.*

Exécuter et observer

Compiler et exécuter le programme **`fork2_wait.c`** dans lequel le père attend la terminaison de son fils.
Qu'observez-vous ?

Exécuter et observer

Compiler et exécuter le programme **`fork2_waitpid.c`**.
Noter et comprendre l'utilisation de la fonction **`waitpid`**.

Exécuter et observer

Modifier le programme **`fork2_wait.c`** en supprimant l'appel à la fonction **`wait`**.
Compiler, exécuter et observer.
Que remarquez-vous ? Comment expliquer un tel comportement ?

5 Code de retour des processus fils

À SAVOIR : fonction `waitpid` et macros de manipulation du code de retour

La primitive `waitpid` est définie comme suit :

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *ptrStatut, int options);
```

Le paramètre `pid` permet de sélectionner le processus dont on attend la fin [a].

- si `pid` > 0, attendre la fin du processus fils de numéro `pid`
- si `pid` $= -1$, attendre la fin d'un processus fils quelconque

La valeur renvoyée est -1 en cas d'erreur, et autrement le numéro du processus fils (zombi) effectivement pris en compte. Si l'erreur est causée par un paramètre `pid` incorrect, (c'est-à-dire quand il n'existe pas de fils correspondant à la demande), le code d'erreur `ECHILD` est assigné à la variable globale `errno`.
L'entier `ptrStatut` donne des informations sur l'état du processus terminé. Pour l'interpréter, il est préférable d'utiliser des fonctions prédéfinies, par exemple :

- `WIFEXITED(ptrStatut)` retourne vrai si le fils s'est terminé normalement (c'est-à-dire s'il ne s'est pas terminé en réaction à l'arrivée d'un signal [b]), faux sinon.
- `WEXITSTATUS(ptrStatut)` donne le statut de sortie (code de retour) du fils, *uniquement si la primitive précédente a renvoyé vrai.*

a. Il y a d'autres possibilités liées aux groupes de processus, non examinées ici.
b. Les *signaux* sont des mécanismes de communication entre processus. Ils seront détaillés ultérieurement.

Exécuter et observer

Compiler, exécuter et observer le programme **`waitpid1.c`**.

Comprendre l'utilisation de la fonction **`waitpid`** et des macros de manipulation de statut de terminaison.

Que fait **`waitpid`** quand en lui passe -1 en premier argument ?

Exercice

Modifier le programme **`waitpid1.c`** pour que le processus principal attende et récupère les informations de terminaison de ses fils dans l'ordre de leur création.

Exécuter et observer

Compiler le programme **`fork_zombi.c`**.

Lancer en arrière plan avec la commande **`./fork_zombi &`**

Regarder les processus avec la commande **`ps`**

Que constatez-vous ?

Tuer les processus en utilisant la fonction **`kill`**.

Explication

Vous devriez observer l'apparition de processus zombis.

Exécuter et observer

Compiler et exécuter le programme **`waitpid_ordre.c`**.

Exécuter plusieurs fois en regardant bien l'ordre des affichages des différents processus.

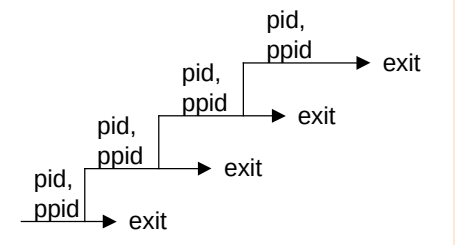
Observez-vous toujours le même ordre ?

Explication

Vous devriez observer des exécutions où les affichages sont entrelacés ce qui montre que l'exécution des différents processus n'est pas ordonnée de manière déterministe i.e. ne suit pas toujours le même ordre.

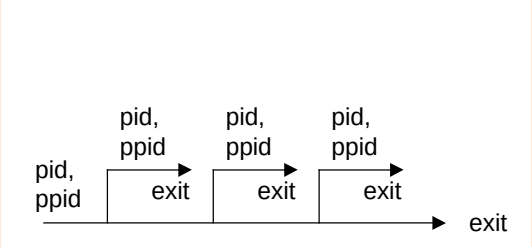
Exercice : Création d'un arbre de processus

Écrire un programme qui crée un arbre de processus comme illustré ci-dessous. La profondeur de l'arbre (sur l'exemple 4) est donnée en paramètre en ligne de commande.



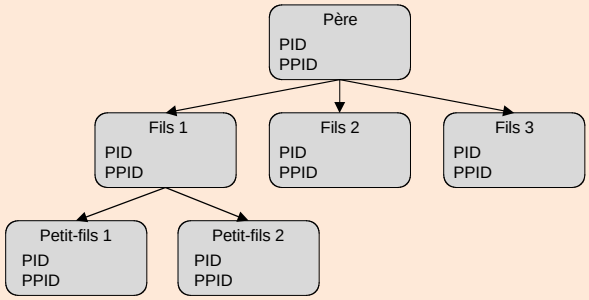
Exercice : Création d'une chaîne de processus

Écrire un programme qui crée une chaîne de processus comme illustré ci-dessous. La largeur de la chaîne (sur l'exemple 4) est donnée en paramètre en ligne de commande.



Exercice : Création d'un arbre généalogique

Écrire un programme qui reproduit l'arbre généalogique représenté ci-après. Chaque processus doit afficher son `PID`, son `PPID`, et afficher les fils qu'il engendre.



Exercice : Valeur de retour de `wait`

Écrire un programme qui crée un fils et qui appelle **`wait`** pour attendre sa terminaison. Quelle est la valeur de retour de **`wait`** ?

Rajouter un deuxième appel à **`wait`**. Quelle est la valeur de retour de ce deuxième appel ?

Exercice : Valeur de retour de `waitpid`

Écrire un programme qui crée deux fils et qui appelle **`waitpid`** pour attendre leur terminaison. Quelle est la valeur de retour de **`waitpid`** ?

Exercice : Valeur de retour de `waitpid`

Écrire un programme qui crée un fils et qui appelle ensuite deux fois **`wait`**. Quelle est la valeur de retour du deuxième appel à **`wait`** ?